

AULA 7

Autenticação com JWT



OBJETIVOS DA AULA



AO FINAL DESSA AULA, VOCÊ VAI SER CAPAZ DE...

- Diferenciar autenticação de autorização
- Entender o que é um JWT e como ele viaja numa requisição
- Fazer hash de senhas com bcrypt, sem nunca guardar texto puro
- Entender o papel do Passport e das Strategies no NestJS
- Entender o papel de um Guard na proteção de rotas



AUTENTICAÇÃO VS. AUTORIZAÇÃO



DUAS PERGUNTAS DIFERENTES

Autenticação "Quem é você?"

Verificar identidade — normalmente com login e senha, validando se as credenciais batem com um usuário cadastrado.

Autorização "O que você pode fazer?"

Depois de saber quem é o usuário, decidir se ele tem permissão para aquela ação ou recurso.

O NEST NÃO TEM AUTH EMBUTIDA

- O Nest se integra com o Passport — a biblioteca de autenticação mais usada no ecossistema Node
- Isso acontece através do pacote `@nestjs/passport`
- Passport funciona com "strategies": cada uma sabe validar um tipo de credencial (usuário/senha, JWT, OAuth...)
- Este curso usa a strategy JWT — token-based, sem sessão guardada no servidor



BCRYPT E O FLUXO DO JWT



NUNCA GUARDE SENHA EM TEXTO PURO

```
const hash = await bcrypt.hash(
  senhaDigitada,
  10, // rounds (custo)
);

const bateu = await bcrypt.compare(
  senhaDigitada,
  hashSalvoNoBanco,
);
```

bcrypt gera um hash unidirecional, com salt embutido — impossível reverter pro texto original.

O CAMINHO DO TOKEN

- 1. Cliente envia usuário/senha em POST /auth/login
- 2. Servidor valida (bcrypt.compare) e assina um JWT com uma chave secreta
- 3. Cliente guarda o token e envia Authorization: Bearer <token> nas próximas requisições
- 4. Servidor valida a assinatura — sem precisar consultar o banco a cada requisição



PASSPORT E GUARDS



JWTSTRATEGY

```
class JwtStrategy extends
  PassportStrategy(Strategy) {

  constructor() {
    super({
      jwtFromRequest:
        ExtractJwt.fromAuthHeaderAsBearerToken(),
      secretOrKey: process.env.JWT_SECRET,
    });
  }

  validate(payload) {
    return { userId: payload.sub };
  }
}
```

GUARD PROTEGENDO UMA ROTA

```
@UseGuards(JwtAuthGuard)
@GetMapping('me')
getProfile(@Request() req) {
  return req.user;
}
```

- Guard decide se a requisição pode prosseguir, antes do handler rodar
- Sem token válido, o Nest já devolve 401 sozinho
- O retorno de validate() vira request.user no controller

